RescueSched for Severity and Deadline Aware Disaster Image Inference on CPU GPU Systems

Sidharth Monangi, Naganishanth Kumar, and Bram Micky
Computer Science and Engineering
Vellore Institute of Technology, India
23BCE2209, 23BCE2223, 23BCE2231

Abstract - Disaster image analysis can involve small urgent requests arriving alongside large surveys on a machine shared by a CPU and a GPU. Fixed execution choices and arrival order queues can delay important requests, while large GPU batches trade response time for throughput. We describe RescueSched, a research prototype that connects application priority to heterogeneous inference scheduling. Each request receives an initial score from responder urgency, location importance, a source severity indicator, and normalized waiting time. For larger requests, an initial subset is classified using the same vision backbone, and probability weighted hazard severity updates the priority of images still awaiting dispatch. A random forest trained on measured execution profiles estimates candidate runtimes from device, execution mode, batch size, resolution, background load, and free GPU memory. A heuristic cost combines predicted completion time, priority weighted deadline overrun, delay imposed on waiting requests, and resource pressure to select dispatch actions. The implementation supports CPU sequential and parallel execution, GPU batches of 1 to 64 images, and batching across compatible requests. We specify evaluation with real AIDER v1 imagery on a single Colab T4 system, fixed and adaptive baselines, and controlled ablations. This report describes the implementation and evaluation protocol; comparative scheduling gains remain to be measured.

Index Terms - Disaster imagery, semantic priority, deadline scheduling, heterogeneous inference, CPU GPU scheduling, dynamic batching, runtime prediction.

I. Introduction

Aerial imagery can show fires, floods, damaged buildings, and blocked routes. EmergencyNet studies efficient aerial classification for drone based emergency monitoring [1], and AIDER supplies real emergency and normal images [2]. Incoming data can mix short responder requests with large surveys. On one CPU GPU host, their order and grouping determine how long potentially important information waits for analysis.

A GPU can process many images together, but a large batch can delay a small request. The CPU provides another execution option when GPU waiting or transfer overhead dominates. The scheduling problem therefore depends on workload and device state. A configuration with high throughput need not provide the shortest response time for an urgent request.

We describe RescueSched, a Python prototype that combines metadata, image estimated severity, measured runtime profiles, resource observations, and deadlines. It couples request ranking with device, execution mode, and batch size selection. The proposed contribution is this integration of staged severity feedback with resource and deadline aware dispatch. The report specifies its implementation and the controlled comparisons needed to establish any scheduling benefit.

II. Related Work and Research Scope

A. Disaster Image Classification

EmergencyNet addresses aerial classification with an efficient convolutional architecture [1]. AIDER contains fire or smoke, flood, collapsed building or rubble, traffic accident, and normal imagery [2]. These works supply application context and real data. RescueSched uses an existing backbone and documented label mapping to study scheduling.

B. Inference Serving and Runtime Scheduling

Clipper separates model serving from training frameworks and uses caching, batching, and adaptive model selection [3]. Nexus schedules DNN workloads across a GPU cluster and coordinates batching and resource use [4]. InferLine combines profiling, simulation, planning, and scaling to configure prediction pipelines under latency constraints [5]. Clockwork builds a serving system around predictable inference execution and request latency objectives [6]. These systems establish that runtime prediction, device allocation, and batching are existing systems techniques.

Orloj models execution time distributions for dynamic DNNs whose runtimes depend on their inputs [7]. RescueSched uses a fixed backbone; its runtime model represents batch size, resolution, execution mode, and resource load rather than input dependent neural execution.

NVIDIA Triton's dynamic batcher already supports request priorities, bounded batching delays, and queue timeouts [8]. Priority queues and dynamic batching are therefore established capabilities. Our focus is the feedback from screened image content to unfinished request priority, coupled with CPU GPU action selection and a priority sensitive evaluation objective.

C. Research Question and Contribution

We ask whether staged semantic information combined with resource aware dispatch improves priority sensitive deadline outcomes on a single CPU GPU machine. The prototype specifies two stage priority, its integration into action scoring and batching, and an evaluation protocol. The reviewed literature establishes relevant foundations; a broader literature and patent search is still required.

III. System Model and Architecture

A request contains one or more images, an arrival time, a relative deadline, an image resolution, responder urgency,

location importance, and a source severity indicator. The latter three metadata values lie in the interval zero to one. The absolute deadline is arrival time plus the relative deadline budget. Candidate inference resolutions are 224, 384, and 512 pixels per side.

The execution host has a CPU worker and, when CUDA is available, a GPU worker. At most one batch is active per device, while the two devices may work concurrently. Scheduling is nonpreemptive: a new request cannot interrupt a batch already executing. Reordering takes effect when an available worker receives a new batch. Fig. 1 shows the information flow and severity feedback loop.

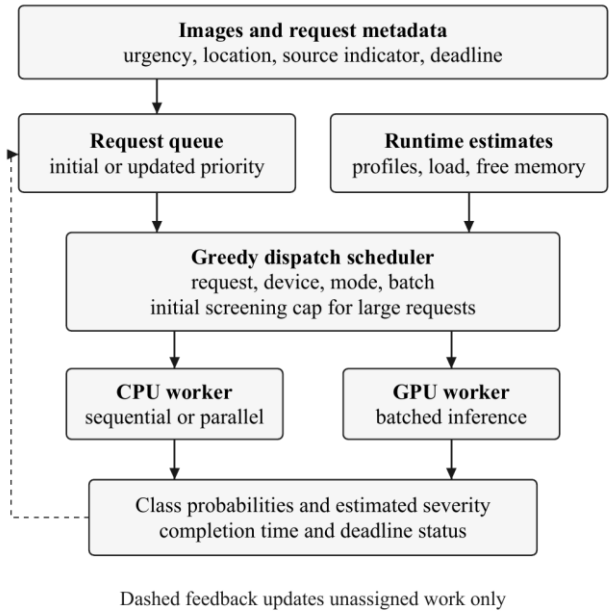


Fig. 1. Implemented scheduling loop. Image classification and measured completion update the state used by later dispatches.

The software contains a dataset adapter, training and inference code, a profiler, a runtime regressor, an event driven engine, and experiment runners. A Flask dashboard accepts requests, displays the queue and device actions, compares predicted and measured batch time, and reads saved experiment summaries. Resource monitoring operates online; profiling and regressor training are separate preparation steps.

IV. Scheduling Method

A. Image Model and Severity Representation

The default backbone is MobileNetV3 Large [9], fine tuned from ImageNet weights through a five class head. Training supports class weighted cross entropy and label smoothing. Table I maps AIDER v1 to the existing interface. Traffic accidents represent an operational hazard proxy rather than a structural damage grade.

TABLE I
AIDER SOURCE MAPPING AND CONFIGURED SEVERITY

AIDER source	Project label	Weight
Normal	no_damage	0.00
Traffic accident	moderate_damage operational hazard proxy	0.50
Flood	flood	0.70
Fire or smoke	fire	0.85
Collapsed building or rubble	severe_damage	1.00

Image severity is the sum of class probabilities multiplied by the configured weights. Request severity is the mean over processed images. These scores are operational proxies; classifier confidence and estimated severity are distinct quantities, and neither provides a calibrated measure of emergency risk.

B. Priority Before and After Inference

Before an image prediction is available, the preliminary priority is computed from urgency U, location importance L, source indicator S, and normalized waiting W. Waiting is elapsed request age divided by its deadline budget, clipped to the interval zero to one. The default score is

$$P_i^{pre} = 0.40U_i + 0.20L_i + 0.25S_i^{src} + 0.15W_i \quad (1)$$

Once some images have been processed, the model estimate enters the score with mixing weight lambda equal to 0.4:

$$P_i^{post} = 0.40P_i^{pre} + 0.60\hat{S}_i \quad (2)$$

The first dispatch of a request with at least 16 images is capped at four screening images and receives a temporary priority floor of 0.6. Screening uses the same classifier. Completions update priority for images still unassigned; concurrent dispatch can limit how much work remains. A single image request has no remaining classification work after prediction, so its initial service order depends on metadata and runtime state.

C. Runtime Profiles and Prediction

Pre run profiling measures preprocessing, host to device transfer, model inference, total service time, and peak GPU allocation. The full profiling design covers all candidate device modes at three resolutions and four external contention settings: none, CPU, GPU, and both. A separate process generates contention so that it resembles another application competing for resources. Each shape is warmed before measurement.

A random forest regressor [10] fits logarithmic batch time using device, mode, padded batch size, resolution, approximate model FLOPs, background CPU and GPU load, and free GPU memory. Defaults are 150 trees, maximum depth 16, and minimum leaf size two. The model is fixed during a run while live readings change its inputs. A random 25 percent holdout estimates interpolation error; withheld configuration and contention groups provide stronger generalization tests.

D. Candidate Actions and Dispatch Cost

The action set contains CPU sequential execution of up to two images, CPU execution with parallel image loading for up to eight images, and GPU batches with capacities 1, 2, 4, 8, 16, 32, and 64. Parallel loading and GPU batches may draw compatible images from several requests at the same resolution. A GPU batch is padded to the next supported power of two, so useful image count and executed tensor size are logged separately.

For each candidate, the system estimates completion delay from the device's remaining busy time and the service time of the request's remaining batches. This is a heuristic forecast under the current state, rather than a guarantee about future arrivals or contention. The action cost is

$$J(i,a) = \alpha\hat{T}_{i,a} + \beta V_{i,a} + \gamma H_{i,a} + \delta R_a \quad (3)$$

Here T is estimated completion delay; V is the deadline overrun penalty; H approximates the priority weighted delay imposed on other waiting requests; and R combines background device load with GPU memory pressure. The default coefficients are alpha = 1, beta = 25, gamma = 0.6, and delta = 0.02. They are configurable heuristic weights, with no claim of optimal calibration. If F is predicted absolute finishing time and D is the deadline, the penalty is

$$V_{i,a} = P_i \, max(0, \hat{F}_{i,a} - D_i)^2 \quad (4)$$

The waiting penalty multiplies estimated device occupation by the summed priorities of other unassigned requests, with a reduction for requests fully absorbed into the same batch. GPU actions are excluded when profiled memory demand exceeds 90 percent of observed free memory. Because memory readings and profile estimates can be imperfect, this check is a feasibility estimate rather than a formal memory safety bound.

E. Greedy Planning and Batching

At arrivals, completions, and periodic replanning, pending requests are ranked by descending priority with deadline and arrival tie breakers. At most the top 12 requests are scored per planning pass. For each request, the least cost feasible action is selected; if its preferred device is busy, the request can wait while another request uses an idle device. This couples request order and execution choice without solving a globally optimal schedule. Low priority GPU work may be held for up to 30 ms when additional batching appears compatible with its deadline.

V. Experimental Protocol

A. Real Data and Hardware

The planned evaluation uses real AIDER v1 images [2], Table I's mapping, a single Colab T4 GPU, and its associated CPU. The manifest builder partitions each class into approximately 65 percent training, 15 percent validation, and 20 percent test images with a fixed seed. Model selection uses training and validation data; scheduling uses held out test images. Archive exact counts and file lists, check duplicates, and recognize that a random class split does not establish event independence.

Record GPU, CPU allocation, free memory, threads, software versions, checkpoint, resolution, and seeds. Profile each Colab session using at least five repetitions per configuration. Report per class precision, recall, F1, macro F1, and a confusion matrix because overall accuracy can hide normal class imbalance.

B. Policies and Workloads

Table II defines the implemented policy set. Every policy must receive the same saved arrival trace, metadata, image list, checkpoint, contention conditions, and workload seed. Workloads include small interactive requests, large surveys, bursts, mixed priority traffic, tight and relaxed deadlines, GPU contention, mixed resolutions, and hidden severity surveys. The last scenario gives similar metadata to surveys whose image contents differ, making the severity feedback mechanism observable.

TABLE II
IMPLEMENTED POLICIES AND CONTROLLED DIFFERENCES

Policy	Decision rule
FixedCPU	FIFO; CPU parallel loading, up to eight images.
FixedGPU	FIFO; GPU batches with fixed capacity 16.
GPUFirst	FIFO; use GPU when free, otherwise CPU.
Threshold	Static image count threshold selects CPU or GPU.
FIFOAdaptive	Runtime based actions; no semantic priority, deadline penalty, screening, or batch hold.
RescueSched S1	Metadata priority and deadline cost; no model severity feedback or initial screening.
RescueSched	Staged severity feedback, deadline cost, and adaptive dispatch and batching.

Arrival intensities and deadline budgets are calibrated to measured service capacity so that a scenario has interpretable load on the host. Real execution is the primary evidence; profile driven simulation supplies additional seeds and parameter sweeps. Synthetic image generation and synthetic backend predictions are suitable for software checks and must be reported separately from real data and hardware measurements.

C. Metrics and Experimental Attribution

Request latency is finish time minus scheduled arrival time, including engine waiting, preprocessing, transfer, and inference. It excludes the dashboard browser's upload and network transfer. Report the mean, 95th and 99th percentiles, images processed per second, deadline violation rate, CPU and GPU busy fractions, memory use, and decision overhead. Unfinished requests count as deadline violations.

Priority weighted deadline violation rate (PWDVR) is computed with a fixed reference weight q for each request:

$$PWDVR = \frac{\sum_i q_i \, I(F_i > D_i)}{\sum_i q_i} \quad (5)$$

The reference weight combines normalized metadata importance with mean label derived severity using the same

0.4 and 0.6 mixing coefficients. These weights are fixed across policies and excluded from the scheduler's inputs; they are evaluation proxies rather than observed harm. Critical image latency uses the configured label severity threshold of 0.75. With Table I, fire or smoke and collapsed building or rubble exceed that threshold; flood at 0.70 does not. Threshold and weight sensitivity must therefore be reported.

Runtime predictor error is measured in milliseconds and percentage error on held out profile samples and dispatched batches. Scheduler decision agreement (SDA) records whether an action matches a measured time oracle for the same lead request and cost function. It does not measure classifier accuracy, global schedule optimality, or the correctness of the entire request order.

A full versus FIFOAdaptive comparison changes several factors and supports only a whole policy comparison. Full versus S1 changes both model severity feedback and screening. To isolate severity mixing, add a matched screening control with identical screening and dispatch constraints but metadata only scores; separately disable the deadline term and batch hold. Repeat real traces with at least five workload seeds, report dispersion and paired differences, and inspect whether reduced critical latency trades off against survey completion, throughput, or fairness.

VI. Prototype Status and Limitations

The software implements data preparation, training, CPU and CUDA inference, profiling, runtime prediction, scheduling policies, real and simulated experiment engines, and the dashboard. No archived classifier metrics, predictor errors, or complete real hardware policy comparisons were available for this report. Numerical accuracy, speedup, and deadline improvement therefore remain unestablished; an interactive request demonstration supports functional execution only.

The proposed mechanism has several limits. Severity is averaged over processed images and can dilute a critical image within a large request. An initial sample can miss rare hazards, and model errors can propagate into priority. Stage 2 can change classification order only while images remain unassigned. The CPU GPU cost model is greedy and uses estimated rather than guaranteed completion times; resource samples may be stale. Waiting age has a bounded influence and supplies no formal starvation bound.

AIDER categories and fixed severity weights do not directly measure within event damage intensity, and the traffic accident mapping is only a scheduling proxy. Generalization requires additional events or datasets and validation of operational severity assumptions. The present scope is one host with one GPU, not distributed or multi node scheduling. Deployment claims would require testing with real operational arrival patterns and domain review of the priority inputs.

The proposed contribution is a specific integration of established building blocks. A library search should examine patent and non patent literature on semantic priority feedback, sample based screening, heterogeneous inference allocation, and deadline sensitive batching. The abstract names these mechanisms to support retrieval of closely related work.

VII. Conclusion

RescueSched connects responder metadata and image estimated severity to CPU GPU inference dispatch on a single heterogeneous host. Its staged priority, profiled runtime model, and deadline cost define an implementable hypothesis about protecting important requests under mixed traffic. The next research step is repeated real hardware evaluation with matched controls, archived traces, and sensitivity analysis of screening and severity assumptions. Those measurements will determine whether the proposed integration yields a useful scheduling advantage.

References

[1] C. Kyrkou and T. Theocharides, "EmergencyNet: Efficient Aerial Image Classification for Drone-Based Emergency Monitoring Using Atrous Convolutional Feature Fusion," IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 13, pp. 1687-1699, 2020. doi: 10.1109/JSTARS.2020.2969809.

[2] C. Kyrkou, "AIDER (Aerial Image Dataset for Emergency Response Applications)," version 1.0, Zenodo, Jun. 2020. doi: 10.5281/zenodo.3888300.

[3] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A Low-Latency Online Prediction Serving System," in Proc. 14th USENIX NSDI, 2017, pp. 613-627. [Online]. Available: USENIX NSDI 2017 paper page.

[4] H. Shen et al., "Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis," in Proc. 27th ACM SOSP, 2019, pp. 322-337. doi: 10.1145/3341301.3359658.

[5] D. Crankshaw et al., "InferLine: Latency-Aware Provisioning and Scaling for Prediction Serving Pipelines," in Proc. ACM SoCC, 2020, pp. 477-491. doi: 10.1145/3419111.3421285.

[6] A. Gujarati et al., "Serving DNNs like Clockwork: Performance Predictability from the Bottom Up," in Proc. 14th USENIX OSDI, 2020, pp. 443-462. [Online]. Available: USENIX OSDI 2020 paper page.

[7] P. Yu, Y. Qiu, X. Jin, and M. Chowdhury, "Orloj: Predictably Serving Unpredictable DNNs," arXiv:2209.00159, 2022. doi: 10.48550/arXiv.2209.00159.

[8] NVIDIA, "Batchers," Triton Inference Server User Guide. Accessed: Oct. 29, 2026. [Online]. Available: NVIDIA Triton documentation.

[9] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF ICCV, 2019, pp. 1314-1324. [Online]. Available: CVF Open Access paper page.

[10] L. Breiman, "Random Forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001. doi: 10.1023/A:1010933404324.